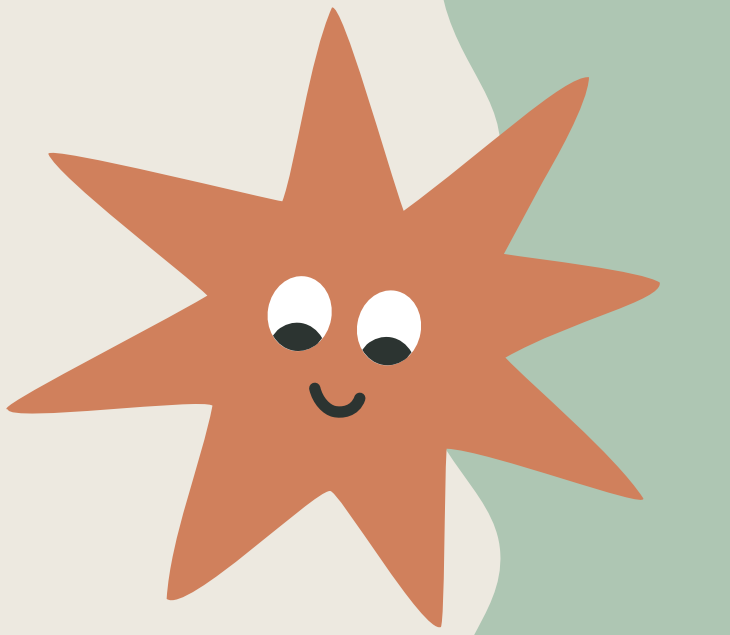
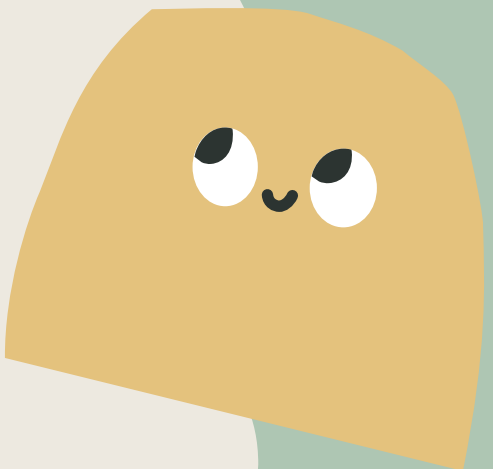
# IN1020

Gruppe 3 – Justas Baltrukonis



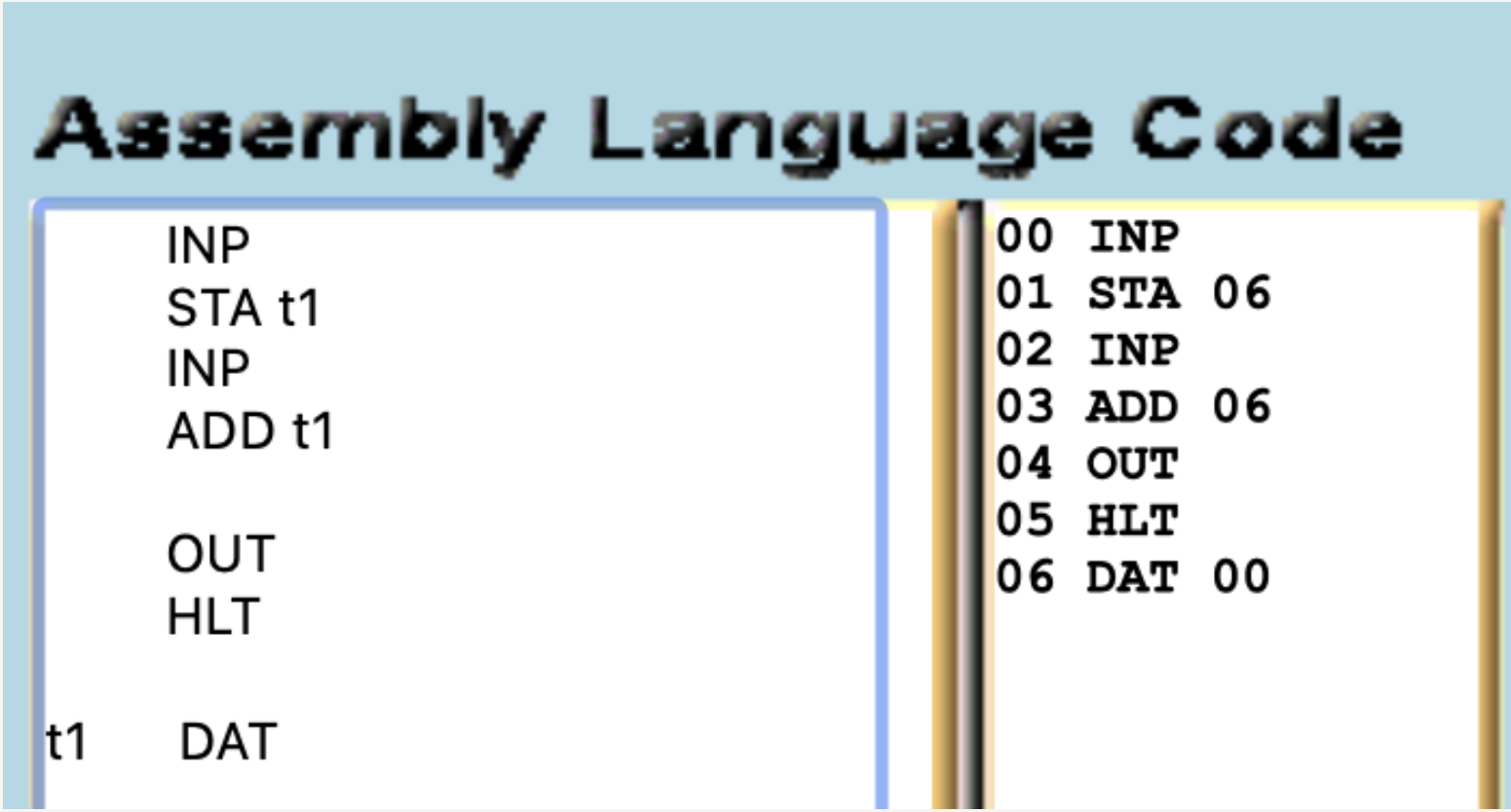


# Kode sammen – oppgave 1

- Legg sammen 2 tall
- INP: 2, 2
- OUT: 4

| Kode | Navn | Hva den gjør |
|---|---|---|
| 0xx | HLT | Stopper kjøringen |
| 1xx | ADD | Legger verdien i postkasse xx til akkumulatoren |
| 2xx | SUB | Trekker verdien i postkasse xx fra akkumulatoren |
| 3xx | STA | Lagrer akkumulatoren i postkasse xx |
| 4xx | – | Ikke i bruk |
| 5xx | LDA | Henter verdien i postkasse xx til akkumulatoren |
| 6xx | BRA | Hopper til adresse xx |
| 7xx | BRZ | Hopper til xx hvis akkumulatoren er 0 |
| 8xx | BRP | Hopper til xx hvis akkumulatoren er 0 eller større |
| 901 | INP | Leser et tall fra innkurven til akkumulatoren |
| 902 | OUT | Skriver akkumulatoren til utkurven |
| 922 | OTC | Skriver akkumulatoren som ett ASCII-tegn |
| – | DAT | Setter av en postkasse til data |

# Kode sammen – oppgave 1



# Oppgave 2

- Gange sammen to tall
- INP: 3, 3
- OUT: 9

| Kode | Navn | Hva den gjør |
|---|---|---|
| 0xx | HLT | Stopper kjøringen |
| 1xx | ADD | Legger verdien i postkasse xx til akkumulatoren |
| 2xx | SUB | Trekker verdien i postkasse xx fra akkumulatoren |
| 3xx | STA | Lagrer akkumulatoren i postkasse xx |
| 4xx | – | Ikke i bruk |
| 5xx | LDA | Henter verdien i postkasse xx til akkumulatoren |
| 6xx | BRA | Hopper til adresse xx |
| 7xx | BRZ | Hopper til xx hvis akkumulatoren er 0 |
| 8xx | BRP | Hopper til xx hvis akkumulatoren er 0 eller større |
| 901 | INP | Leser et tall fra innkurven til akkumulatoren |
| 902 | OUT | Skriver akkumulatoren til utkurven |
| 922 | OTC | Skriver akkumulatoren som ett ASCII-tegn |
| – | DAT | Setter av en postkasse til data |

# Oppgave 2

## Assembly Language Code

```
        INP
        STA t1
        INP
        STA t2


løkke   LDA tot
        ADD t1
        STA tot


        LDA t2
        SUB en
        STA t2


        BRZ ferdig
        BRA løkke

// alternativ
        // BRP løkke
        // SUB t1



ferdig  LDA tot
        OUT
        HLT


en      DAT 1
t1      DAT
t2      DAT
tot     DAT
```

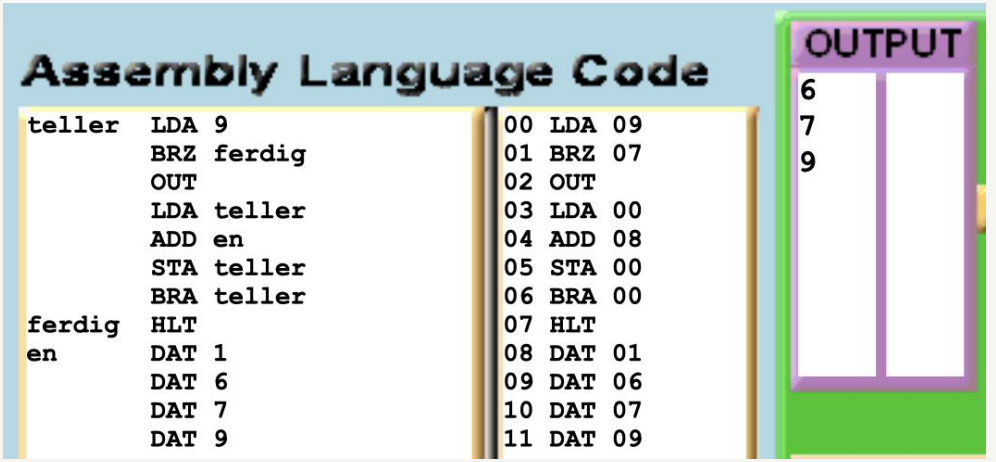
```
00 INP
01 STA 16
02 INP
03 STA 17
04 LDA 18
05 ADD 16
06 STA 18
07 LDA 17
08 SUB 15
09 STA 17
10 BRZ 12
11 BRA 04
12 LDA 18
13 OUT
14 HLT
15 DAT 01
16 DAT 00
17 DAT 00
18 DAT 00
```

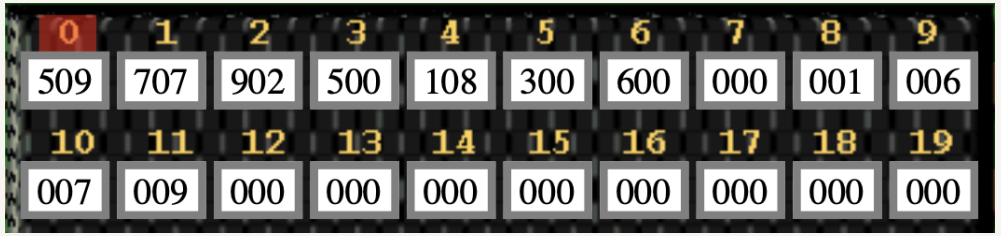
# Andre oppgaver ?

| Kode | Navn | Hva den gjør |
|---|---|---|
| 0xx | HLT | Stopper kjøringen |
| 1xx | ADD | Legger verdien i postkasse **xx** til akkumulatoren |
| 2xx | SUB | Trekker verdien i postkasse **xx** fra akkumulatoren |
| 3xx | STA | Lagrer akkumulatoren i postkasse **xx** |
| 4xx | – | Ikke i bruk |
| 5xx | LDA | Henter verdien i postkasse **xx** til akkumulatoren |
| 6xx | BRA | Hopper til adresse **xx** |
| 7xx | BRZ | Hopper til **xx** hvis akkumulatoren er 0 |
| 8xx | BRP | Hopper til **xx** hvis akkumulatoren er 0 eller større |
| 901 | INP | Leser et tall fra innkurven til akkumulatoren |
| 902 | OUT | Skriver akkumulatoren til utkurven |
| 922 | OTC | Skriver akkumulatoren som ett ASCII-tegn |
| – | DAT | Setter av en postkasse til data |

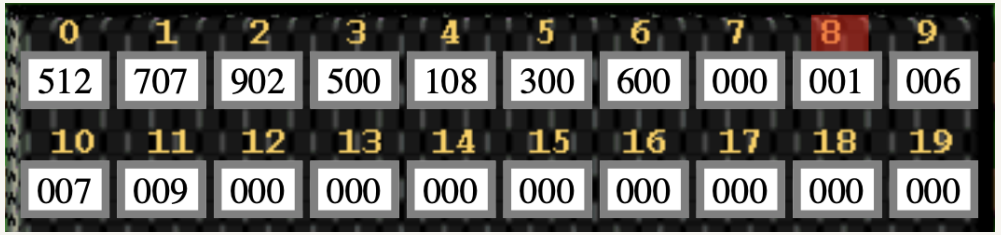
# Selvmodifiserende kode

- Hvordan?
  - Dere vet sikkert om tags til løkker, men denne taggen kan forandres!
  - Disse tagsa har dere sett man bruker til BRA og BRZ
  - Men man kan også bruke andre!
    - LDA, STA
  - Man kan derfor modifisere denne koden!
  - Von Neumann-arkitekturen – ingen fysisk forskjell på data og kode
- Kode eksempel til høyre



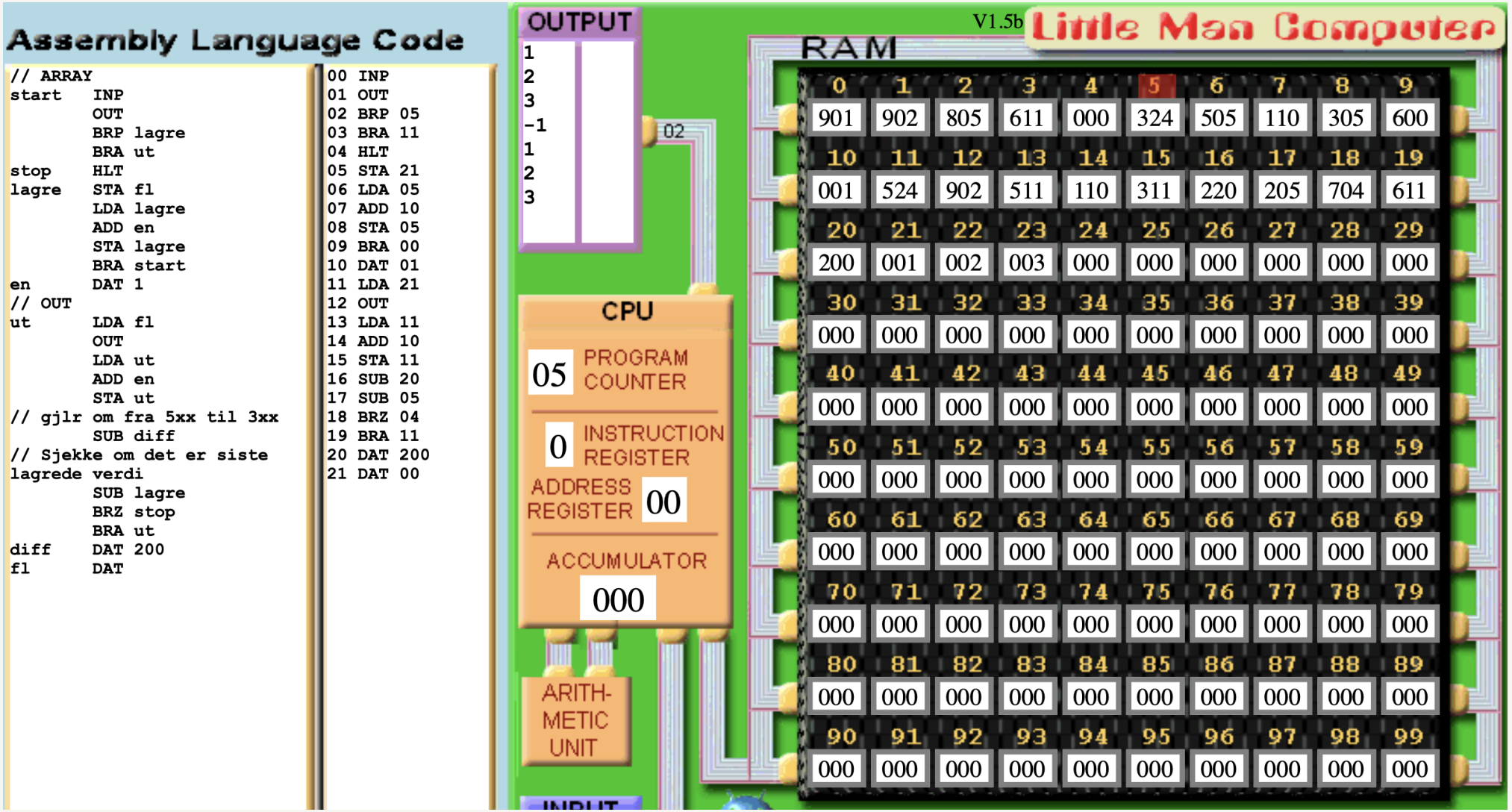




- LDA + tag er egentlig bare tall!




- Takeaway?
  - Tags er nice for løkker, men kan brukes med alt!

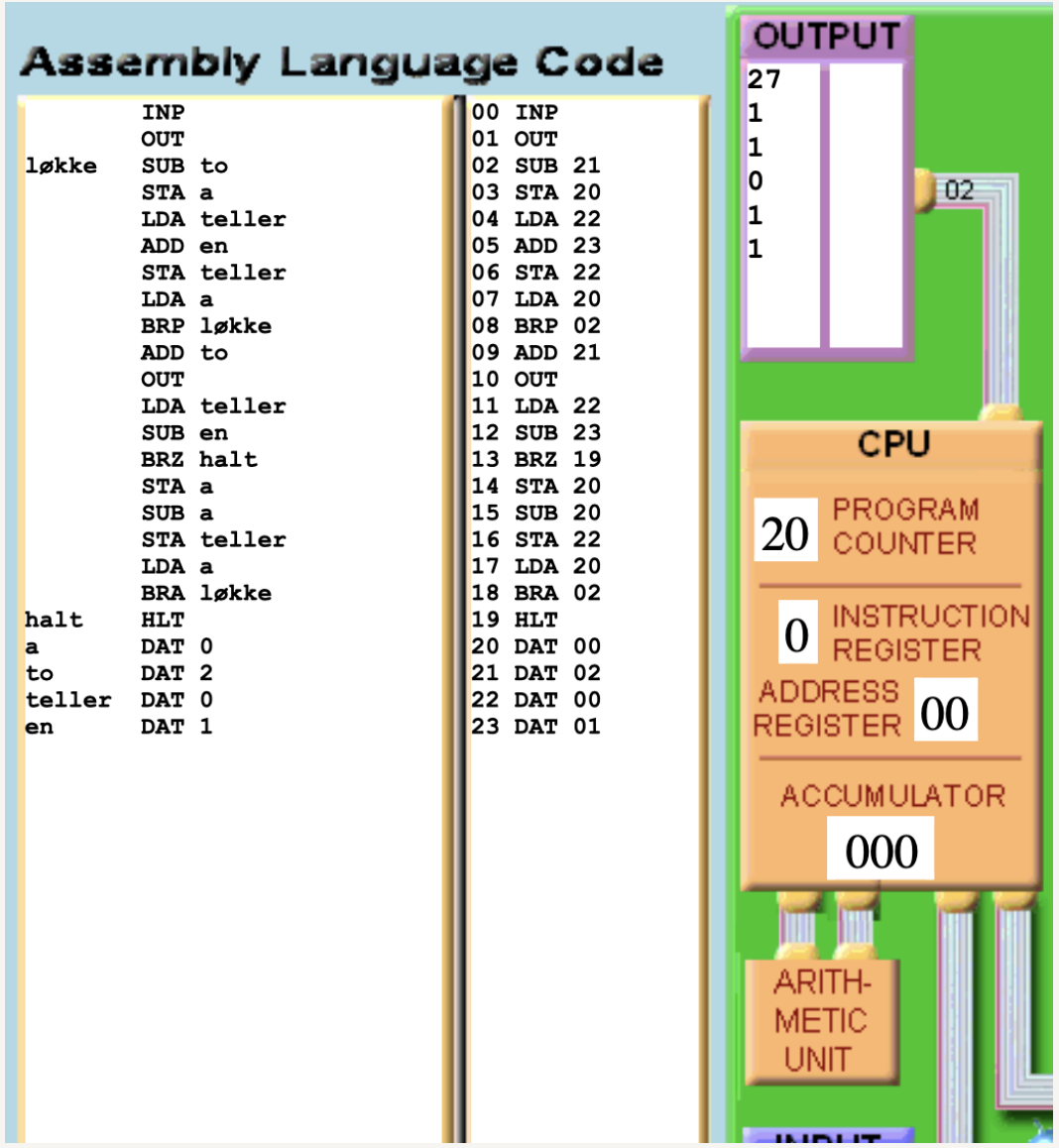
# Kult program



# Binærkalkulator LMC (Oblig 1 2023)

- Dette er oblig 1 i 2023
- Koden har en binærkalkulator.
  - Den skriver ut tallet i desimal
  - Deretter skriver den ut i binær
- Binærtallet leses nedenfra og opp
- Koden følger samme prinsipp og modulo akkurat som omregning fra desimal til binær omregning med deling og modulo.

- Flyt:
  - Tar minus to til enten -1 eller -2 (modulo hadde gitt 0 eller 1, man får 0 og 1 etter å legge til 2)
  - Legger til 2 (vet om det er 0 eller ikke, dermed har man første verdi)
    - Variasjon mellom om teller eller a er i akkumulator
  - A blir satt til neste tall (halvparten i heltall)
  - Teller blir 0
  - loop



# Uendelig hva man kan gjøre (nesten)

- Minnet er begrenset
- Likevel kan man oppnå mye med selvendrende kode!

# Ukesoppgaver og universet – oblig – se hva man får til i LMC!